

FRI commitment scheme — design and code map

This package implements a polynomial commitment scheme based on **FRI** (Fast Reed–Solomon Interactive Oracle Proof of Proximity), with **DEEP-ALI** combining for batched openings, **k-way coset folding** with coset-per-leaf Merkle layout, and optional **proof-of-work grinding** for soundness amplification.

The implementation is configurable but the defaults follow the recipe used by ethSTARK / Plonky-3: `BlowupFactor = 2`, `FoldingFactor = 8`, `FinalPolynomialMaxLen = 16`, `NumQueries = 20`. The protocol parameters live on `fri.Config` and can be overridden per-application.

This document explains the math, then maps each step onto specific code with file references.

1. Notation

Symbol	Meaning
$\mathbb{F}$	the prime field (<code>gnark-crypto/koalabear</code>)
n	base-domain size of an input polynomial (must be a power of two)
ρ	Reed–Solomon rate = $1 / \text{BlowupFactor}$ (default $1/2$)
N	codeword-domain size, $N = n/\rho = \text{BlowupFactor} \cdot n$
L	the codeword domain $\langle \omega \rangle \subset \mathbb{F}^*$ (order N)
ω	a primitive N -th root of unity, generator of L
k	the folding factor (<code>FoldingFactor</code> , default 8)
ζ	a primitive k -th root of unity, $\zeta = \omega^{N/k}$
α_ℓ	the verifier's folding challenge at FRI layer ℓ
β	the DEEP-ALI combiner challenge
x_{ij}, y_{ij}	evaluation point and claimed value of the j -th open request on oracle i
$Q_i(X)$	the i -th oracle's DEEP quotient polynomial
$q(X)$	the β -weighted sum of all Q_i — the input to FRI
g_ℓ	the layer- ℓ codeword ($g_0 = q$ on L)
m	number of query positions per round, <code>Config.NumQueries</code>

All polynomials are stored in **Lagrange-normal form** (evaluations on a domain of the form $\langle \omega \rangle$, in natural index order).

2. The recipe at a glance

We are given some number of polynomials $f_1, \dots, f_K$ (potentially split across multiple `Commit` calls), each in Lagrange-normal form on its own base domain of size n_i . We want to convince a verifier of two things:

1. **Proximity.** Each f_i is close to a codeword of $\text{RS}[\mathbb{F}, L, n_i]$ — informally, "each f_i really is a low-degree polynomial".
2. **Evaluation.** A list of claims of the form " $f_i(z) = y$ " all hold.

The protocol does this in five phases:

- **Phase A — RS commit.** Each oracle is RS-encoded on the **common** codeword domain L of size N (the same N for every oracle in a single `Prove` call). A coset-per-leaf Merkle tree commits to the encoded values.
- **Phase B — DEEP combiner.** For each oracle f_i with open requests at points $x_{i1}, \dots, x_{iR_i}$ with claimed values $y_{i1}, \dots, y_{iR_i}$, the prover forms one DEEP quotient $Q_i(X) = (f_i(X) - I_i(X)) / \prod_j (X - x_{ij})$, where I_i is the Lagrange interpolant of the claims about f_i . After a Fiat-Shamir challenge β , the per-oracle quotients are combined into a single codeword $q = \sum_i \beta^i Q_i$ on L .
- **Phase C — k-way FRI folding.** q is repeatedly folded by a factor of k using transcript-derived challenges α_ℓ , until the codeword length drops to $\leq \text{FinalPolynomialMaxLen}$. The remaining short codeword is sent in full.
- **Phase D — Optional grinding + queries.** After binding the final polynomial, the prover finds a 64-bit nonce that hashes (with the transcript state) to a value with `Config.GrindingBits` leading zero bits. Then m query indices are drawn from the transcript.
- **Phase E — Verification.** The verifier replays Fiat-Shamir, re-derives every challenge, and per query checks (i) every Merkle path; (ii) that the locally reconstructed q -coset matches the layer-0 coset; and (iii) that each fold step is consistent across layers all the way to `FinalPolynomial`.

The Reed–Solomon rate ρ together with m and `GrindingBits` determine soundness; see §10.

3. File map

File	Role
fri.go	Config, Commitment, OpeningProof, Committer (commit phase), Verifier, Bind, RegisterOpenAt, ClaimedValueAt, Prove, coset-Merkle helpers
fold.go	k-way FRI fold (foldCoset, foldLayer, elementPow)
deep.go	DEEP combiner (computeClaimedValue, buildDEEPCombiner, computeBarycentricWeights, accumulateDEEP)
query.go	Building per-query Merkle openings + coset data
verify.go	Verifier.VerifyOpening — full proof check
grind.go	Proof-of-work helpers (grindAndBind, verifyAndBindGrinding)
fri_test.go	Unit tests; see also export_test.go for test exports

Reused infrastructure (not in this package):

- internal/merkle — Merkle tree (merkle.New, BuildIthLeaf, OpenProof, Verify).
- internal/reedsolomon — Encoder.Encode (Lagrange-on-base → Lagrange-on-codeword via FFT round trip).
- internal/poly — poly.Evaluate (Lagrange interpolation at an arbitrary point).
- gnark-crypto/ fiat-shamir — the transcript.

4. Configuration

Config collects every protocol parameter:

```

type Config struct {
    BlowupFactor      int    //  $p^{-1}$ ; default 2
    FoldingFactor      int    //  $k$ ; default 8
    FinalPolynomialMaxLen int    // stop folding when  $|g| \leq$  this; default 16
    NumQueries         int    //  $m$ ; default 20
    MaxCodewordDomainSize uint64 // unify codeword domains across Commits
    GrindingBits        int    // PoW bits; default 0 = off
}

```

`applyDefaults` ([fri.go:60-73](#)) fills zero fields with the defaults declared as `DefaultFRI*` constants ([fri.go:21-25](#)).

`MaxCodewordDomainSize` exists because the codeword size N must be known at the **first** `Commit` call. `Commit` immediately RS-encodes, builds the Merkle tree, and binds the root to the transcript; the next round's challenge derivation depends on the bound root, so we cannot defer encoding to `Prove` time and then auto-derive N from the largest oracle seen. Committing only over the un-encoded base domain (size n_i) would eliminate redundancy and break proximity testing, so encoding must happen up front.

When multiple `Commit` calls cover polynomials of different base sizes, the FRI protocol requires every oracle share **one** codeword domain so that a single q can combine all DEEP quotients. Setting `MaxCodewordDomainSize = BlowupFactor · max( $n_i$ )` forces the encoder to lift all oracles to that size. The check in `Commit` is

```

if c.config.MaxCodewordDomainSize > codewordDomainSize {
    codewordDomainSize = c.config.MaxCodewordDomainSize
}

```

([fri.go:188-190](#)). The reverse check — that all oracles agree on N — is enforced at the start of `Prove`.

`GrindingBits` is the only parameter the verifier needs to know externally (it's not encoded in the proof — see §11).

5. Phase A — RS encoding and coset-per-leaf Merkle layout

5.1 Reed-Solomon encoding

A polynomial p given in Lagrange form on a domain of size n is encoded on L of size $N = \text{BlowupFactor} \cdot n$ by going through canonical form:

$$\text{Lagrange}[n] \xrightarrow{\text{IFFT}_n} \text{canonical}[n] \xrightarrow{\text{zero-pad}} \text{canonical}[N] \xrightarrow{\text{FFT}_N} \text{Lagrange}[N]$$

This is exactly what `reedsolomon.Encoder.Encode` does. `Committer.encode` ([fri.go:495-509](https://fri.go.495-509)) memoises `fft.Domain` objects per base size so polynomials with different n_i in the same `Commit` batch don't repeatedly rebuild domains.

5.2 Coset-per-leaf Merkle layout

For folding factor k , the next layer's domain is the image of L under $x \mapsto x^k$. The k preimages of a single next-layer point form an **arithmetic progression** of indices in L :

If $i = j + t \cdot (N/k)$ for any $t \in 0, \dots, k-1$ and fixed $j \in [0, N/k]$, then $(\omega^i)^k = \omega^{kj+tN} = \omega^{kj}$ (because $\omega^N = 1$). So all k preimages share the same k -th power.

A naive Merkle tree that puts one codeword position per leaf would force k separate Merkle proofs per query (one per preimage). Putting all k preimages in **one leaf** collapses these into a single proof, dropping query communication and verifier work by a factor of k .

`buildCosetMerkleTree` constructs this layout:

```
leaf_j = LeafHash( poly_0[j], poly_0[j+N/k], ..., poly_0[j+(k-1)·N/k],
                  poly_1[j], ...,
                  poly_{K-1}[j+(k-1)·N/k] )
```

— K polynomials per oracle, k positions per polynomial, fixed deterministic order.

For internal FRI **layers** (commit-phase intermediate codewords), there's only one polynomial per layer, so `buildLayerMerkleTree` uses k field elements per leaf instead of $K \cdot k$.

The verifier reconstructs leaf bytes via `cosetLeafBytes` (prover and verifier share this helper, which is a one-line wrapper over `marshalElements`).

5.3 Auto-DEEP open per oracle

After the Merkle commitment is bound to the transcript, `Commit` draws one DEEP point z per polynomial in the batch, by computing a separate challenge `<challengeName>@deep_<polyName>`:

```
deepBytes, _ := c.transcript.ComputeChallenge(deepChallengeName(challengeName, name))
deepPt.SetBytes(deepBytes)
c.Open(name, deepPt)
```

(fri.go:235-247). This populates `c.openRequests` (one per polynomial per `Commit`).

`Verifier.Bind(challengeName, commitment)` mirrors `Committer.Commit`'s transcript operations in this exact order (fri.go:443-481):

1. `NewChallenge` once per polynomial in the commitment (for each polynomial's auto-DEEP point) and once for the commitment itself.
2. `Bind(challengeName, root)` — Merkle root of the coset-per-leaf tree.
3. For each polynomial: `Bind(deepChallengeName, root)`, then `ComputeChallenge(deepChallengeName)` → DEEP point z , recorded in `v.deepPoints`.
4. `ComputeChallenge(challengeName)` — derives the post-commitment Fiat-Shamir challenge value consumed by the next round of the loom prover.

Callers may also register additional evaluation claims via

`Verifier.RegisterOpenAt(name, point)` (see §12 for the full API and §13 for the loom integration); these must be called in the same order as the corresponding prover-side `Committer.Open` calls so that `deepPoints` indices line up with `ClaimedValues`.

6. Phase B — DEEP combiner

6.0 What "DEEP-ALI" means

DEEP = "Domain Extension for Eliminating Pretenders" (Ben-Sasson, Goldberg, Kopparty, Saraf 2019). An out-of-domain opening at $z \notin L$ collapses the Reed–Solomon decoding list to a single polynomial, removing the ambiguity that proximity-only testing would leave.

ALI = "Algebraic Linking IOP" (ethSTARK §5). The β -weighted linear combination of multiple low-degree claims into one — what the per-oracle Q_i below participates in. Together, DEEP-ALI converts a list of evaluation claims into a single FRI proximity test.

6.1 Math — per-oracle merged form

For oracle i opened at R_i points $x_{i1}, \dots, x_{iR_i}$ with claimed values $y_{i1}, \dots, y_{iR_i}$, define the per-oracle DEEP quotient

$$Q_i(X) = \frac{f_i(X) - I_i(X)}{\prod_j (X - x_{ij})},$$

where I_i is the unique polynomial of degree $< R_i$ satisfying $I_i(x_{ij}) = y_{ij}$ for all j . Because $f_i(x_{ij}) - I_i(x_{ij}) = 0$, the numerator vanishes at every x_{ij} , so Q_i is a polynomial (no remainder). The combined DEEP codeword is

$$q(X) = \sum_i \beta^i Q_i(X).$$

The requirement $z \notin L$ for every x_{ij} ensures no denominator is zero on L (the codeword domain).

Partial-fractions identity. Rather than computing I_i explicitly and performing an interpolation FFT, we use

$$\frac{I_i(X)}{\prod_k (X - x_{ik})} = \sum_j \frac{w_{ij}}{X - x_{ij}}, \quad w_{ij} = \frac{y_{ij}}{\prod_{k \neq j} (x_{ij} - x_{ik})}.$$

This gives a pointwise formula for Q_i at any $\omega^m \in L$:

$$Q_i(\omega^m) = \frac{f_i(\omega^m)}{\prod_k (\omega^m - x_{ik})} - \sum_j \frac{w_{ij}}{\omega^m - x_{ij}}.$$

Computing $Q_i(\omega^m)$ for all $m = 0, \dots, N-1$ costs $O(R_i^2 + R_i N)$ field operations per oracle, with all inversions batched via `koalabear.BatchInvert`.

6.2 Code walk-through

`buildDEEPCombiner`:

1. Groups `openRequests` by `polyKey{oracleI, name}`, preserving first-opened order.
2. Sorts `order` canonically by `(oracleI, name)` so that β -powers are independent of `open` call order.
3. Per oracle: precomputes w_{ij} via `computeBarycentricWeights`, then batch-inverts the product denominators $\prod_k (\omega^m - x_{ik})$ and the per-pole denominators $(\omega^m - x_{ij})$ for all m and j in two `BatchInvert` calls.
4. `accumulateDEEP` adds $\beta^i \cdot Q_i(\omega^m)$ to $q[m]$ for $m = 0, \dots, N-1$.

Edge cases: `deepCombineErr` if any $x_{ij} \in L$; `deepDuplicateErr` if the same polynomial is opened twice at the same point.

The claimed values $y_{ij} = f_i(x_{ij})$ are computed by `computeClaimedValue` (Lagrange interpolation on the codeword domain) and stored in `OpeningProof.ClaimedValues` in registration order, matching `openRequests`.

6.3 Why one β suffices for many oracles

Prove enforces a single shared codeword domain via the size check at [fri.go:285-298](#). Once every oracle lives on the same L , the β -combination is a single proximity statement — there is no soundness penalty from batching, only one extra Schwartz-Zippel factor in β . The β -powers now index **oracles** (not individual open requests), so one β -exponent covers all of a polynomial's evaluation claims at once.

This is **method 1 (degree padding)** from Boneh's whiteboard treatment: extend everyone to a common degree before combining. We chose this over **method 2 (independent FRI per oracle)** because the loom AIR backend expects to commit many small chunks (one per AIR-quotient piece) and method 1 amortises the FRI fold cost across them.

7. Phase C — k -way FRI folding

7.1 The split

Any polynomial f of degree $< d$ can be uniquely decomposed by powers of X modulo k :

$$f(X) = f_0(X^k) + X \cdot f_1(X^k) + \cdots + X^{k-1} \cdot f_{k-1}(X^k), \quad \deg(f_j) < d/k.$$

For a folding challenge α , define the k -way fold

$$\mathrm{fold}_\alpha(f)(Y) := \sum_{j=0}^{k-1} \alpha^j \cdot f_j(Y).$$

$\deg(\mathrm{fold}_\alpha(f)) < d/k$. This is the next layer's polynomial.

7.2 Pointwise computation on a coset

Fix $x_0 \in L$. The k preimages of $Y := x_0^k$ under $x \mapsto x^k$ are

$\zeta^t \cdot x_0 : t = 0, \dots, k-1$ where ζ is a primitive k -th root of unity. Then:

$$f(\zeta^t \cdot x_0) = \sum_j (\zeta^t \cdot x_0)^j \cdot f_j(Y) = \sum_j \zeta^{tj} \cdot (x_0^j \cdot f_j(Y)).$$

Reading the right-hand side as a function of t : `cosetValues[t]` are the DFT of $(x_0^j \cdot f_j(Y))_j$ over the k -th roots of unity. Inverting the DFT recovers $c_j := x_0^j \cdot f_j(Y)$, and then

$$\text{fold}_\alpha(f)(Y) = \sum_j \alpha^j \cdot f_j(Y) = \sum_j (\alpha/x_0)^j \cdot c_j.$$

So one fold step at one coset costs **one inverse DFT of size k plus one Horner evaluation at α/x_0** .

7.3 Code

`foldCoset`:

```
copy(coeffs, cosetValues)
kDomain.FFTInverse(coeffs, fft.DIF) // (1) IDFT over the k-th roots
utils.BitReverse(coeffs)           //      -> coeffs[j] = x_0^j · f_
j(Y)

invBase.Inverse(&cosetBase)          // (2) compute α / x_0
evalAt.Mul(&alpha, &invBase)

// (3) Horner: result = Σ_j coeffs[j] · evalAt^j
for i := k-1; i >= 0; i-- {
    result.Mul(&result, &evalAt)
    result.Add(&result, &coeffs[i])
}
```

`foldLayer` calls `foldCoset` for each of the N/k coset bases $x_0 = \omega^j$, $j \in [0, N/k)$, producing the next layer's codeword.

`runFRICommitPhase` is the loop:

```

for len(g) > config.FinalPolynomialMaxLen && len(g) >= k {
    layerTree := buildLayerMerkleTree(g, k, ...)
    transcript.Bind("fri@layer_<i>", root)
    α := transcript.ComputeChallenge("fri@layer_<i>")
    g = foldLayer(g, α, gen, k)
    gen = elementPow(gen, k)    // ω_{ℓ+1} = ω_ℓ^k
}

```

Termination: each iteration shrinks g by a factor of k , so the loop runs at most $\lceil \log_k(N/\text{FinalPolynomialMaxLen}) \rceil$ times.

7.4 The “no folding happens” edge case

If $N \leq \text{FinalPolynomialMaxLen}$ on entry, the loop body never executes — `finalCodeword = g = q`. This is a legitimate configuration (small input, large `FinalPolynomialMaxLen`) and the verifier handles it specially; see §9.4. The standalone test `TestRoundTripNoLayers` exercises this branch directly.

8. Phase D — Final polynomial, grinding, and queries

After `runFRICommitPhase` returns, `Prove` does three things in order, each separated by a transcript bind so each step's input depends on all previous ones:

1. **Bind the final polynomial.** [fri.go:340-348](#) — the small `finalCodeword` is serialised and bound under `"fri@final"`.
2. **Grinding (optional, §10).** [fri.go:350-358](#) — when `GrindingBits > 0`, find a nonce satisfying the PoW constraint and bind it.
3. **Derive query indices.** [fri.go:361-366](#) — `deriveQueryIndices` draws m challenges `"fri@query_<i>"` and masks each into $[0, N/k)$.

Then `buildQueryProofs` walks each query position once and emits:

- For each oracle: a Merkle proof of the leaf at j plus the $K \cdot k$ coset values (`OracleOpenings`, `OracleCosetData`).

- For each layer ℓ : a Merkle proof of the leaf at $j_\ell = j \bmod (N_\ell/k)$ plus the k coset values (`LayerOpenings`, `LayerCosetData`).

The leaf-index recurrence between layers is

$$j_{\ell+1} = j_\ell \bmod (N_{\ell+1}/k)$$

implemented in [query.go:106-110](#) as `jEll = jEll % nLeavesNext`.

9. Phase E — Verification

`Verifier.VerifyOpening` replays every Fiat-Shamir operation the prover did, then runs per-query checks. It is **self-describing** about every protocol parameter except `GrindingBits` (see §11).

9.1 Replay

The verifier first re-derives every challenge in the same order the prover used:

1. β (from `"fri@combine"`) — [verify.go:53-61](#).
2. α_ℓ (one per `LayerCommitment`, by binding the root and computing `"fri@layer_<ℓ>"`) — [verify.go:64-79](#).
3. The final-polynomial bind under `"fri@final"` — [verify.go:81-99](#).
4. Grinding (if `GrindingBits > 0`): replay the same seed / nonce dance as the prover via `verifyAndBindGrinding`; this both verifies the PoW *and* keeps the transcript in sync — [verify.go:101-106](#).
5. Query indices via `deriveQueryIndices`; the verifier compares them byte-for-byte against `proof.QueryIndices` — [verify.go:108-118](#).

If anything diverges from the prover's transcript order, the indices won't match and verification rejects. This makes Fiat-Shamir mismatches a single-line failure rather than a downstream Merkle failure.

9.2 Inferring k

The proof is **self-describing** about k :

```
switch {
case numLayers > 0 && len(LayerCosetData) > 0:
    k = len(LayerCosetData[0][0])
case len(OracleCosetData) > 0:
    K := Commitments[0].NumPolynomials
    k = len(OracleCosetData[0][0]) / K
default:
    return error
}
```

([verify.go:36-58](#)). The verifier never reads `Config.FoldingFactor`.

9.3 Per-query check (layered path)

For each query at leaf index $j \in [0, N/k)$:

a. DEEP quotient reconstruction. The verifier groups `v.deepPoints` by (`oracleI`, `name`) — the same grouping `buildDEEPCombiner` uses — and reconstructs the combined q -values at the k coset positions $\omega^{j+t \cdot N/k}$, $t = 0, \dots, k-1$:

$$\text{qCheck}[t] = \sum_i \beta^i \cdot Q_i(\omega^{j+t \cdot N/k}),$$

where each Q_i is evaluated using the same partial-fractions formula as the prover:

$$Q_i(\omega^m) = \frac{f_i(\omega^m)}{\prod_k (\omega^m - x_{ik})} - \sum_j \frac{w_{ij}}{\omega^m - x_{ij}}.$$

The values $f_i(\omega^m)$ come from `OracleCosetData`; the y_{ij} needed to form weights w_{ij} come from `ClaimedValues` (in registration order). [verify.go:131-235](#).

b. Oracle Merkle paths. Each oracle's path is verified once per query (not once per request) — the same coset bytes serve every claimed open from that oracle. [verify.go:238-249](#).

c. q vs layer-0. `qCheck[t]` must equal `LayerCosetData[qi][0][t]`. [verify.go:270-275](#).

d. Fold consistency across layers. For each layer ℓ :

- Verify the Merkle proof against `LayerCommitments[l]`.
- Compute `cosetBase =  $\omega_\ell^{j_\ell}$` , then `foldCoset(LayerCosetData[qi][l],  $\alpha_\ell$ , cosetBase, kDomain)`.
- For internal layers: this must equal `LayerCosetData[qi][l+1][t_{l+1}]`, where $t_{l+1} = j_\ell / (N_{l+1}/k)$ (the position of j_ℓ within the next layer's coset).
- For the final layer: this must equal `FinalPolynomial[j_l]`.

[verify.go:280-312](#).

The leaf-index recurrence $j_{\ell+1} = j_\ell \bmod (N_{\ell+1}/k)$ mirrors `buildQueryProofs`. The accompanying generator update $\omega_{\ell+1} = \omega_\ell^k$ is done via `elementPow`.

9.4 Per-query check (0-layer path)

When `numLayers == 0`, the codeword was already small enough that no fold ever ran, and `FinalPolynomial` is the entire q . Step (c) is folded directly into the `FinalPolynomial` check, and the fold loop is skipped:

```
for t := range k {
    pos := j + t*nLeaves
    if !qCheck[t].Equal(&proof.FinalPolynomial[pos]) {
        return error
    }
}
```

([verify.go:256-268](#)).

This branch is exercised by `TestRoundTripNoLayers` and `TestNoLayersTamperedFinalPolynomial`.

9.5 What can go wrong, and where it surfaces

Tamper class	Failure surface
Flipped oracle codeword value	<code>qCheck</code> mismatch <i>and</i> Merkle path mismatch (steps a/b).
Tampered claimed value	<code>qCheck</code> mismatch (step a).
Tampered final polynomial	Fold mismatch at last layer or 0-layer direct check.
Tampered <code>LayerCommitment</code> root	Re-derived α_ℓ differs $\rightarrow$ Merkle/ <code>qCheck</code> cascade fails.
Tampered Merkle sibling	<code>merkle.Verify</code> returns false.
Tampered query index	<code>derivedIndices</code> mismatch (verify.go:113-117).
Tampered grinding nonce	<code>verifyAndBindGrinding</code> rejects, or transcript diverges.
z lands on L	Inversion in DEEP rebuild fails (verify.go:211-213).

Every failure mode has a test in `fri_test.go`; see the "Tampered..." suite.

9.6 On-domain opens via coset FFT (future work)

The current code encodes all codewords on L itself. In principle, opens at $z \in L$ could be supported by encoding on a coset $g \cdot L$ of L (gnark-crypto supports `fft.WithCoset()`). The evaluation domain shifts away from L , keeping every denominator non-zero. Switching is invasive — the Merkle layout, fold-domain progression, and verifier DEEP reconstruction would all need updating — and is unnecessary for loom: what might look like

"open column A at row 0" is an AIR vanishing constraint, evaluated through the expression DAG at the out-of-domain point ζ , not a direct FRI opening at a domain point.

10. Phase F — Grinding (proof of work)

10.1 Math

Each spot-check query has soundness error $\varepsilon := 1 - \delta$, where δ is the relative Hamming distance to the closest codeword. With $\rho = 1/2$, the proximity-gap heuristic gives $\delta_{\max} \approx 0.29$, so $\varepsilon \approx 0.71$. With m queries the total error is ε^m , and the bit security is

$$\kappa \approx m \cdot \log_2(1/\varepsilon) \approx 0.49 \cdot m.$$

For a target κ , the prover can do PoW work to "buy" b free bits, pushing the required query count down to

$$m = \lceil (\kappa - b) / \log_2(1/\varepsilon) \rceil.$$

For $\kappa = 128$, $b = 24$ reduces m from ≈ 261 to ≈ 213 .

10.2 Construction

The grinding hash is

$H(\text{seed} \parallel \text{nonce}_{\text{be64}})$ — SHA256 in this implementation,

where `seed` is a fresh transcript challenge derived **after** the final polynomial has been bound, but **before** queries are drawn:

```
seed = transcript.ComputeChallenge("fri@grinding_seed")
nonce = smallest n ∈ ℕ such that H(seed || n) has ≥ GrindingBits leading
zeros
transcript.Bind("fri@grinding_nonce", nonce)
```

Binding the nonce back into the transcript ensures the query indices depend on it — a malicious prover cannot grind the seed *and* cherry-pick queries in parallel.

10.3 Code

Helper	Role
<code>grindingHash</code>	<code>SHA256(seed nonce_be64)</code>
<code>hasLeadingZero-Bits</code>	Bit-counting at byte boundaries (covered by <code>TestHasLeadingZeroBitsBoundary</code>)
<code>deriveGrinding-Seed</code>	Pulls the seed from the transcript
<code>bindGrindingNonce</code>	Mirrors prover/verifier transcript bind of the chosen nonce
<code>grindAndBind</code>	Prover-side: search nonce, bind it
<code>verifyAndBind-Grinding</code>	Verifier-side: check the supplied nonce satisfies the bit threshold, then bind

The default `GrindingBits = 0` short-circuits both sides — neither prover nor verifier touches the grinding transcript names, and existing proofs are unaffected.

10.4 Verifier requirement

`GrindingBits` is the **only** parameter the verifier needs externally — unlike `k`, `NumQueries`, etc., it is **not encoded in the proof**. Otherwise a malicious prover could grind 0 bits and claim a strong PoW. The verifier must therefore be configured by its caller via the public field `Verifier.GrindingBits` (or, at the loom integration level, `verifier.WithFRIGrindingBits(n)`).

11. Self-describing proof: what the verifier doesn't need

The proof carries enough information that the verifier infers all of these without external configuration:

Quantity	Inferred from
N (codeword size)	<code>Commitments[0].CodewordDomainSize</code>
K (polys per oracle)	<code>Commitments[oi].NumPolynomials</code>
<code>numLayers</code>	<code>len(LayerCommitments)</code>
<code>numQueries</code>	<code>len(QueryIndices)</code>
k (folding factor)	layer coset length, or oracle coset length / K
<code>FinalPolynomial</code> size	<code>len(FinalPolynomial)</code>

Only `GrindingBits` must be agreed externally (§10.4).

`ClaimedValues` lists one entry per registered open request, in the same order that `Committer.Open` was called (auto-DEEP points from `Commit` first, then any additional `Open` calls). The verifier accesses these through `Verifier.ClaimedValueAt(proof, name, point)` after `VerifyOpening` has validated the proof.

12. Public API surface

This section enumerates every exported symbol callers interact with. All live in package `github.com/consensus/loom/internal/commitment/fri`. Internal helpers (coset Merkle layout, fold step, query proofs, grinding helpers) are not part of this surface.

12.1 Constants and sentinel errors

```
const (
    DefaultFRIBlowupFactor      = 2 //  $p^{-1}$ 
    DefaultFRIFoldingFactor     = 8 //  $k$ 
    DefaultFRIFinalPolynomialMaxLen = 16
    DefaultFRINumQueries        = 20
)

var ErrInvalidPolynomial = errors.New("fri commitment: invalid polynomial")
```

`ErrInvalidPolynomial` is the only exported sentinel; it wraps every malformed input rejection from `Commit` (empty polynomial name, zero-length polynomial, zero codeword domain, codeword domain smaller than the folding factor). Domain-mismatch errors from `Prove` and DEEP precondition failures from `buildDEEPCombiner` are returned as plain `fmt.Errorf` strings.

12.2 Config

```
type Config struct {
    BlowupFactor      int    //  $p^{-1}$ ; default 2
    FoldingFactor     int    //  $k$ ; default 8
    FinalPolynomialMaxLen int    // stop folding when  $|g| \leq$  this; default 16
    NumQueries        int    //  $m$ ; default 20
    MaxCodewordDomainSize uint64 // unify codeword domains across Commits
    GrindingBits      int    // PoW bits; default 0 = off
}
```

Field semantics are detailed in §4. Zero fields are filled by the unexported `applyDefaults`, which `NewCommitter` invokes once at construction time.

12.3 **Commitment** (*oracle metadata*)

```
type Commitment struct {
    Root                []byte
    BaseDomainSize      uint64
    CodewordDomainSize  uint64
    NumPolynomials      int
    PolynomialNames     []string // sorted alphabetically
    PolynomialSizes     []uint64
}
```

One `Commitment` is produced per `Committer.Commit` call. The verifier feeds the matching `Commitment` to `Verifier.Bind` to advance its transcript.

`PolynomialNames` is the sort order used to lay out the Merkle leaves and order the coset data — the verifier never needs to recompute it, only read it.

12.4 **OpeningProof**

```
type OpeningProof struct {
    Commitments          []Commitment
    LayerCommitments     [][]byte
    FinalPolynomial      []koalabear.Element
    ClaimedValues        []koalabear.Element
    QueryIndices         []uint64
    OracleOpenings       [][]merkle.Proof
    OracleCosetData      [][][]koalabear.Element
    LayerOpenings        [][]merkle.Proof
    LayerCosetData       [][][]koalabear.Element
    GrindingNonce        uint64
}
```

This is the wire format. `ClaimedValues[r]` is the prover-claimed evaluation $f_r(x_r)$ for the r -th registered open request, in the order the prover called `Commit` (auto-DEEP first) and `Open` (manual opens after). After `VerifyOpening` returns, callers read individual values back through `Verifier.ClaimedValueAt`.

12.5 Prover side: `Committer`

```
func NewCommitter(
    transcript *fiatshamir.Transcript,
    config      Config,
    leafHasher merkle.LeafHasher,
    nodeHasher merkle.NodeHasher,
) Committer
```

Constructs a fresh committer. `transcript` may be nil for a transcript-less mode (used in low-level tests); in normal use, callers share one transcript across the FRI committer and any outer protocol.

```
func (c *Committer) Commit(challengeName string, polys map[string]poly.
    Polynomial) error
```

RS-encodes every polynomial in `polys`, builds the coset-per-leaf Merkle tree, binds the root to the transcript under `challengeName`, and registers one auto-DEEP open per polynomial. Polynomial names within `polys` must be unique and non-empty.

```
func (c *Committer) Open(name string, point koalabear.Element) error
```

Records an additional out-of-domain evaluation claim. `name` must have been committed by a prior `Commit` call; `point` must lie outside the code-word domain L . The actual evaluation $y = f(\text{point})$ is computed lazily inside `Prove`. Calling `Open` twice with the same `(name, point)` is rejected by `Prove` with `deepDuplicateErr` semantics, so callers should deduplicate.

```
func (c *Committer) Commitment(name string) Commitment
```

Returns the `Commitment` metadata of the oracle that holds `name`. Useful for callers that pass commitments to a separate verifier component.

```
func (c *Committer) Prove() (OpeningProof, error)
```

Closes the protocol: computes claimed values for every registered open, derives the DEEP combiner challenge β , builds q , runs the FRI commit phase (folds, layer Merkle trees), optionally grinds, derives query indices, and assembles per-query Merkle openings. After `Prove` returns the committer is spent — callers should not reuse it.

12.6 Verifier side: `Verifier`

```
type Verifier struct {
    Transcript    *fiatshamir.Transcript
    GrindingBits int
    // unexported: deepPoints, oracleCount
}

func NewVerifier(transcript *fiatshamir.Transcript) Verifier
```

`GrindingBits` must be set by the caller to match the prover-side `Config.GrindingBits`; it is the only protocol parameter not encoded in the proof (§10.4).

```
func (v *Verifier) Bind(challengeName string, commitment Commitment) error
```

Mirrors the prover's `Commit` transcript operations exactly — see §5.3 for the four-step enumeration. Each `Bind` call records one auto-DEEP point per polynomial in `commitment.PolynomialNames` into `v.deepPoints`, in the same order the prover registered them.

```
func (v *Verifier) RegisterOpenAt(name string, point koalabear.Element) error
```

Mirrors a prover-side `Committer.Open(name, point)` call. **Does not touch the transcript** (the prover-side `Open` is also transcript-silent). The name must already appear in some bound oracle. Registration order **must match** the prover's `Open` call order so that `deepPoints` indices line up with `OpeningProof.ClaimedValues`.

```
func (v *Verifier) ClaimedValueAt(  
    proof OpeningProof,  
    name string,  
    point koalabear.Element,  
    ) (koalabear.Element, error)
```

Looks up the prover-claimed evaluation of `name` at `point`. The `(name, point)` pair must have been registered (auto-DEEP via `Bind`, or manual via `RegisterOpenAt`) before `VerifyOpening` was called. Returns an error if the pair is unknown or its registration index is out of range.

Best practice: only call this **after** `VerifyOpening` succeeds — the returned value is only as trustworthy as the FRI verification that bound it.

```
func (v *Verifier) VerifyOpening(  
    proof OpeningProof,  
    lh merkle.LeafHasher,  
    nh merkle.NodeHasher,  
    ) error
```

Runs the full per-query check (§9). The `lh` / `nh` hashers must match the prover-side hashers passed to `NewCommitter`. Returns nil on success; on failure returns a descriptive error pinpointing the failure surface (Merkle mismatch, qCheck mismatch, fold mismatch, query-index divergence, etc. — see §9.5).

12.7 Typical usage flow

```
// Prover
committer := fri.NewCommitter(transcript, cfg, leafHash, nodeHash)
committer.Commit("round_0", polysRound0)
// (more rounds, more Commits...)
committer.Open("colA", zeta)
committer.Open("colA_rot1", omegaZeta)
proof, _ := committer.Prove()

// Verifier (sharing a separately-built transcript)
v := fri.NewVerifier(transcript)
v.GrindingBits = cfg.GrindingBits
v.Bind("round_0", proof.Commitments[0])
// (more Binds in matching order...)
v.RegisterOpenAt("colA", zeta)
v.RegisterOpenAt("colA_rot1", omegaZeta)
if err := v.VerifyOpening(proof, leafHash, nodeHash); err != nil { /* r
eject */ }
yA, _ := v.ClaimedValueAt(proof, "colA", zeta)
yArot, _ := v.ClaimedValueAt(proof, "colA_rot1", omegaZeta)
```

§13 walks through the same flow as embedded inside the loom prover/verifier.

13. Wiring into the loom prover/verifier

The loom-level `prover.Prove` constructs the FRI committer with a config derived from the program ([prover/prover.go](#)):

```
friCfg := fri.Config{
    MaxCodewordDomainSize: fri.DefaultFRIBlowupFactor * maxModuleN,
    GrindingBits:          config.FRIGrindingBits,
}
```

After deriving ζ , the prover calls `Committer.Open(name, evalPoint)` for every AIR-relevant evaluation: first all AIR-quotient chunks (in sorted module name and chunk-index order), then all committed and rotated columns

from each module's vanishing relation (same sorted order, duplicate `(name, evalPoint)` pairs skipped). `Prove()` computes and bundles the resulting claimed values in `OpeningProof.ClaimedValues`.

The loom verifier mirrors this in `deriveChallenges` via `Verifier.RegisterOpenAt` (in the identical deterministic order), and then reads the FRI-verified claimed values back via `ClaimedValueAt` in `populateAIREvaluations` — which runs **after** `VerifyOpening` so that the AIR check only uses values the FRI proof has already certified.

The `config.FRIGrindingBits` value comes from the caller-supplied `prover.WithFRIGrindingBits(n)` option. The verifier mirrors this with `verifier.WithFRIGrindingBits(n)`; both sides default to 0 (no grinding). Tests:

- `TestVerifierWithGrinding` — full prove/verify round-trip with 8-bit PoW.
- `TestVerifierGrindingMismatch` — prover grinds, verifier doesn't, must reject.
- `TestVerifierTamperedClaimedValue` — a claimed value is mutated after proving; FRI verification must reject.

Within `internal/commitment/fri/fri_test.go` the corresponding tests are `TestGrindingRoundTrip`, `TestGrindingTamperedNonce`, `TestVerifierGrindingMismatch`, and `TestVerifierMissingGrinding`.

14. Soundness parameter cheat sheet

Setting	κ (bits) at $\rho = 1/2$	Notes
$m = 20, b = 0$ (defaults)	≈ 9.8	Suitable for tests / development only.
$m = 120, b = 0$	≈ 59	Mid-range, no grinding.
$m = 261, b = 0$	≈ 128	Pure FRI 128-bit conjectured soundness.
$m = 213, b = 24$	≈ 128	Cheaper proof, $2^{24} \approx 16\text{M}$ -hash PoW.
$m = 180, b = 40$	≈ 128	Smaller proof, $2^{40} \approx 10^{12}$ hashes — significant PoW.

These figures use the standard heuristic and assume the proximity gap (ethSTARK §A.6); see Plonky-3 / RiscZero / ethSTARK for more conservative analyses.

15. Limitations and known gaps

- **AIR-side openings** (committed columns at ζ , rotated columns at ω^{shift} . ζ , AIR-quotient chunks at ζ) are wired through FRI's `Open / RegisterOpenAt` API. The loom prover calls `Committer.Open` for every AIR-relevant evaluation after deriving ζ ; the loom verifier mirrors each call with `Verifier.RegisterOpenAt` before `VerifyOpening`. `OpeningProof.ClaimedValues` is the **only** channel from which the verifier reads these values; there is no parallel un-bound data path.
- **Pre-existing vet warning** in `prover/prover.go` (`return copies lock value`) is unrelated to FRI and not introduced by this work; flagged separately.
- **Soundness defaults** (`NumQueries: 20`, `GrindingBits: 0`) are not production parameters; they're sized for fast tests. Pick m and b based on the

security target and proof-size budget for your application.